1. Introduction

This assignment demonstrates how to create a simple interactive Graphical User Interface (GUI) using Java Swing. The application displays a welcome message and an icon. It includes two buttons: Change Text and Change Background. The first button cycles through different messages, and the second button cycles through different background colours. The program uses JFrame, JPanel, JLabel, ImageIcon, constructors, inheritance and event handling.

2. Application Design and Components

JFrame creates the main application window. JPanel holds the interface components. JLabel displays the welcome text and icon. ImageIcon loads the image file. The buttons allow the user to interact with the application.

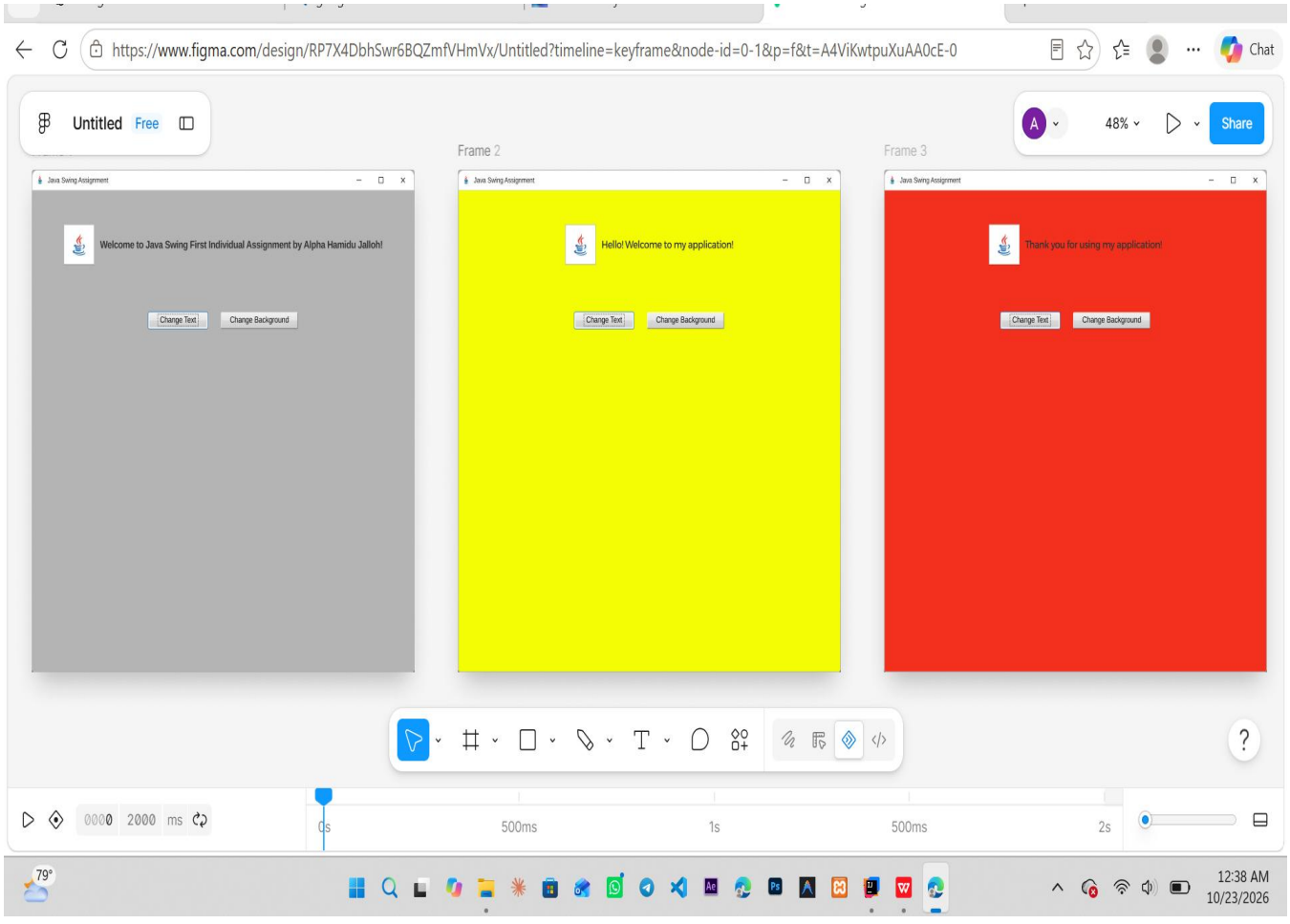


Figure 1: Figma wireframe of the Java Swing application.

3. Program Requirements and Implementation

JFrame: Creates the main window titled "Java Swing Assignment". The window size is set to 800 × 800 pixels.

JPanel: Provides the area where the welcome label and buttons are placed.

JLabel: Displays the welcome message with the icon.

ImageIcon: Loads the image file named icon.png. The code resizes it to 45 × 45 pixels so it does not take over the window.

Constructors: The MyPanel() and SwingAssignment() constructors set up the components when objects are created.

Inheritance: MyPanel extends JPanel, and SwingAssignment extends JFrame.

Event handling: Action listeners make the text and background buttons respond each time they are clicked.

4. Source Code

```
import javax.swing.*;
import java.awt.*;

class MyPanel extends JPanel {

    private JLabel label;
    private int textIndex = 0;
    private int colorIndex = 0;

    private final String[] messages = {
            "Welcome to Java Swing First Individual Assignment by Alpha Hamidu Jalloh!",
            "Hello! Welcome to my application!",
            "This is my Java Swing assignment!",
            "Thank you for using my application!",
            "Keep learning and improving!"
    };

    private final Color[] colors = {
            Color.LIGHT_GRAY,
            Color.CYAN,
            Color.YELLOW,
            Color.PINK,
            Color.ORANGE,
            Color.GREEN,
            Color.RED
    };

    // Constructor
    public MyPanel() {

        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
        setBackground(colors[colorIndex]);
        setBorder(BorderFactory.createEmptyBorder(35, 20, 20, 20));

        // Load the image and resize it to a suitable size
        ImageIcon originalIcon = new ImageIcon("src/icon.png");

        Image resizedImage = originalIcon.getImage()
```

```
                .getScaledInstance(45, 45, Image.SCALE_SMOOTH);

        ImageIcon icon = new ImageIcon(resizedImage);

        // Welcome label with a small icon
        label = new JLabel(messages[textIndex], icon, JLabel.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 16));
        label.setAlignmentX(Component.CENTER_ALIGNMENT);

        add(label);
        add(Box.createVerticalStrut(25));

        // Create the buttons
        JButton textButton = new JButton("Change Text");
        JButton colorButton = new JButton("Change Background");

        textButton.setFont(new Font("Arial", Font.BOLD, 14));
        colorButton.setFont(new Font("Arial", Font.BOLD, 14));

        // Keep the buttons together
        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 12, 5));
        buttonPanel.setOpaque(false);

        buttonPanel.add(textButton);
        buttonPanel.add(colorButton);

        buttonPanel.setAlignmentX(Component.CENTER_ALIGNMENT);
        add(buttonPanel);

        // Change text every time the button is clicked
        textButton.addActionListener(e -> {
            textIndex = (textIndex + 1) % messages.length;
            label.setText(messages[textIndex]);
        });

        // Change background colour every time the button is clicked
        colorButton.addActionListener(e -> {
            colorIndex = (colorIndex + 1) % colors.length;
            setBackground(colors[colorIndex]);
            repaint();
        });
    }
}

public class SwingAssignment extends JFrame {

    // Constructor
    public SwingAssignment() {
        setTitle("Java Swing Assignment");
        setSize(800, 800);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        add(new MyPanel());
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            SwingAssignment app = new SwingAssignment();
            app.setVisible(true);
        });
    }
}
```

5. Testing and Results

The application should be tested to check that the window opens and both buttons work repeatedly.

- The window opens with the title "Java Swing Assignment".
- The icon appears at a suitable size beside the welcome message.
- Clicking Change Text repeatedly changes the message.
- Clicking Change Background repeatedly changes the panel colour.
- Clicking the window close button exits the application.

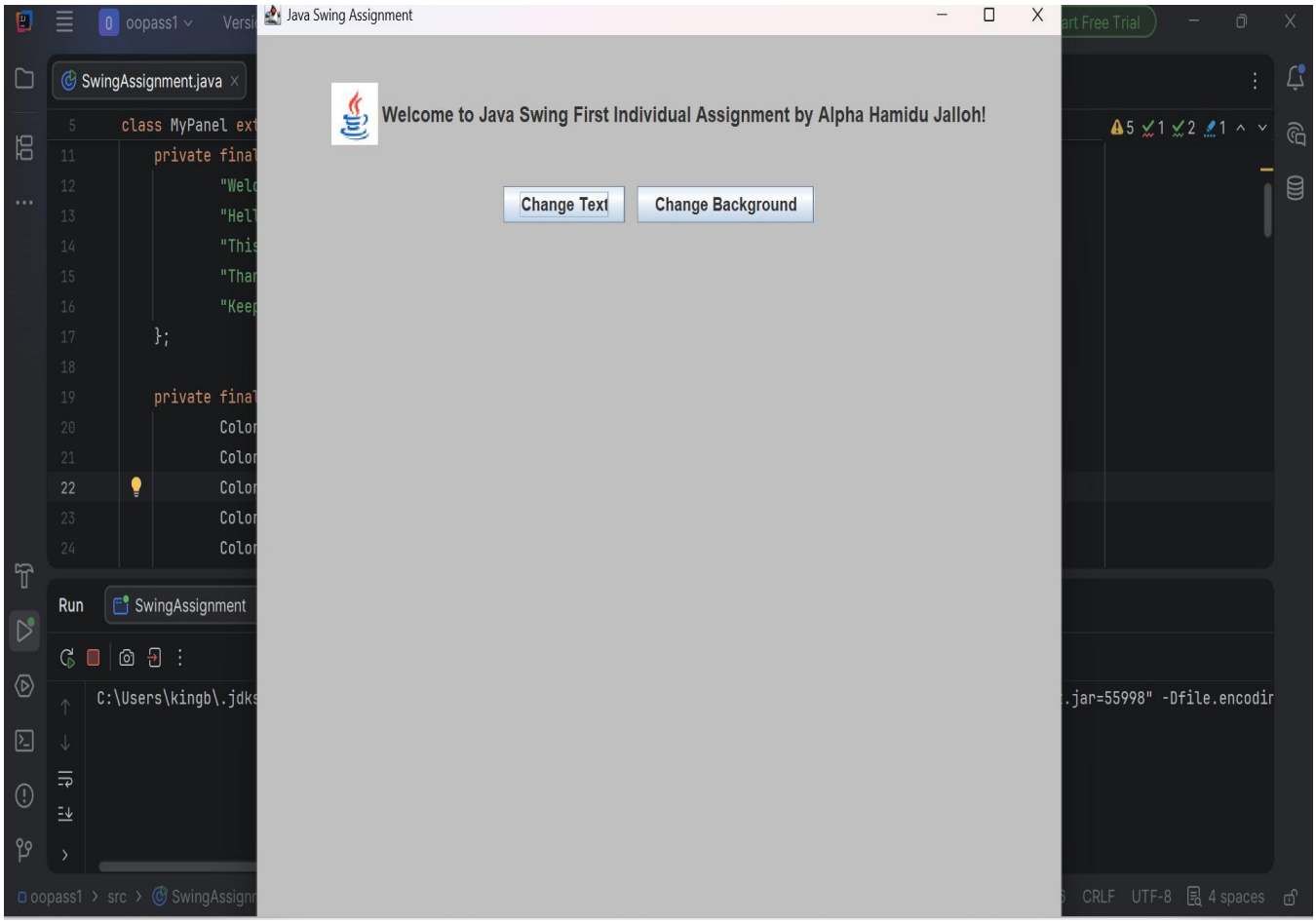


Figure 2: Screenshot of the running Java Swing application.

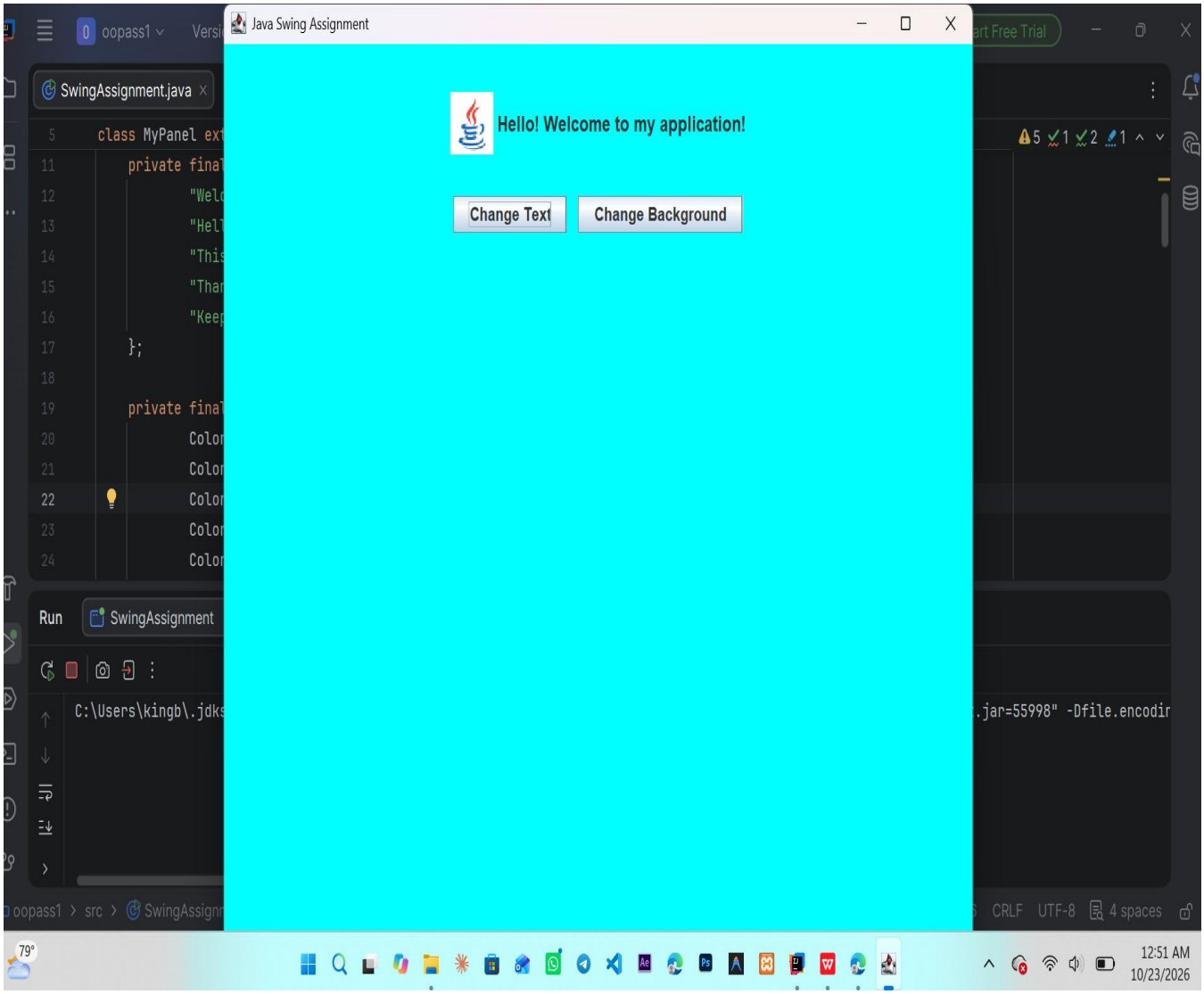


Figure 3: Application after a button has been clicked.

6. Conclusion

The assignment demonstrates a simple interactive Java Swing application. It uses a window, panel, label and image icon, as well as inheritance, constructors and event handling. The two buttons allow the user to change the message and background colour more than once.